

Module 1 — ROOT Primer

Supplementary to the sPHENIX Standalone Course

1. Why ROOT?

ROOT is the analysis framework of choice in high-energy physics. It gives you histograms, trees, fitting, and graphics in one toolkit, with a C++ interpreter (Cling) and a Python binding (PyROOT). Nearly every plot in a collider-physics paper was made with ROOT.

For this course you need three capabilities: read and write **TTree** data, fill and fit **histograms**, and manipulate **TLorentzVector** four-vectors. Everything else is optional.

2. Installation

Binary distribution (fastest)

```
cd $HOME/opt
wget https://root.cern/download/root_v6.30.02.Linux-ubuntu22-x86_64-gcc11.4.tar.gz
tar xf root_v6.30.02.Linux-*.tar.gz
mv root root-6.30.02 && ln -s root-6.30.02 root
source $HOME/opt/root/bin/thisroot.sh
```

Source build (if binaries don't match your distribution)

```
git clone --branch v6-30-02 https://github.com/root-project/root.git
cmake -S root -B root-build -DCMAKE_INSTALL_PREFIX=$HOME/opt/root
cmake --build root-build -j$(nproc) --target install
```

3. A 1-minute tour

Open an interactive ROOT session with `root -l`. The `-l` skips the splash screen.

```
>>> TH1F h("h", "test", 50, -5, 5);
```

```
>>> for (int i=0; i<10000; ++i) h.Fill(gRandom->Gaus());  
>>> h.Draw();  
>>> h.Fit("gaus");  
>>> .q
```

That's a complete workflow: create a histogram, fill it, draw it, fit it. Everything else is variations on that theme.

4. Histograms

The main classes are **TH1F** (1D, 32-bit bins), **TH1D** (1D, 64-bit), **TH2F** (2D), **TProfile** (mean of y in bins of x).

Class	When to use
TH1F / TH1D	1D distribution; use TH1D if bin counts exceed $\sim 2 \times 10^4$
TH1I	Integer-valued; rarely needed
TH2F / TH2D	Correlations (η vs ϕ , E vs p_T)
TProfile	Mean of y at each bin of x — great for $\langle E \rangle$ vs layer
THStack	Overlay several TH1F sharing x-axis

Note: Always set `h.Sumw2()` before filling with event weights — otherwise error bars are computed from bin entries instead of weight sums, and they're wrong.

5. TTree: your data-on-disk

A TTree is a columnar store. Each 'branch' is a typed array and you read events one at a time.

```
// Write
TFile f("out.root", "RECREATE");
TTree tree("events", "minimal demo");
float pt, eta;
tree.Branch("pt", &pt, "pt/F");
tree.Branch("eta", &eta, "eta/F");
for (int i=0; i<1000; ++i) {
    pt = gRandom->Exp(2.0);
    eta = gRandom->Gaus(0, 1.5);
    tree.Fill();
}
tree.Write();

// Read
TFile f("out.root");
```

```
TTree* t = (TTree*) f.Get("events");
float pt, eta;
t->SetBranchAddress("pt", &pt);
t->SetBranchAddress("eta", &eta);
for (Long64_t i=0; i<t->GetEntries(); ++i) {
    t->GetEntry(i);
    if (pt > 1.0) { /* fill histograms */ }
}
```

6. TLorentzVector

Four-vector arithmetic for invariant masses and kinematic quantities:

```
TLorentzVector a, b;  
a.SetPtEtaPhiM(5.0, 0.3, 0.0, 0.0); // (pT, η, φ, m)  
b.SetPtEtaPhiM(4.2, -0.1, 2.5, 0.0);  
double m_ab = (a + b).M();  
double dR   = a.DeltaR(b);
```

Note: In ROOT ≥ 6.30 the modern equivalent is `ROOT::Math::PtEtaPhiMVector` — non-deprecated but less common in legacy code. Either is fine for this course.

7. Fitting with TF1

Fit a Gaussian to a histogram with one line:

```
h.Fit("gaus"); // quick  
TF1 f("f", "[0]*exp(-0.5*((x-[1])/[2])**2)", -5, 5);  
f.SetParameters(100, 0, 1); // init  
h.Fit(&f, "R"); // restricted range
```

8. PyROOT

PyROOT gives you Python access to ROOT classes via the ROOT module:

```
import ROOT  
f = ROOT.TFile.Open("out.root")  
t = f.events  
for ev in t:  
    if ev.pt > 1.0:  
        print(ev.pt, ev.eta)
```

Use PyROOT for quick explorations and interactive plotting; use C++ ROOT for large loops where performance matters (PyROOT has per-event overhead).

9. Recipe: a quick analysis pipeline

A typical analysis in this course follows four steps: read events, select, fill histograms, fit and plot. Keep all four in one macro file so the workflow is reproducible:

```
void analysis() {  
    TFile f("events.root");  
    TTree* t = (TTree*) f.Get("events");  
    TH1F* h = new TH1F("h", "pT", 50, 0, 20);  
    t->Draw("pt>>h", "pt > 0.5");  
    h->Fit("expo");  
    new TCanvas(); h->Draw();  
}
```